This should remind you, once again, how important is to be careful when writing recursive methods and, at the same time, how important is to use the right data structure: while, in theory, treaps could also cause stack overflows, since their height is logarithmic, we would need to add `~215000 = 105000` elements to cause a stack overflow (which is way more than it's possible to allocate on any computer's RAM anyway, so it's unlikely you'll ever get a crash from a recursive treap).

3.5.3 Profiling Running Time

The height of the tree, however, is not the only criteria we are interested in: we want to know if there is a catch, and what's the price we have to pay in terms of running time and memory usage.

To find out this, we have run a proper profiling of the first test (with unbounded integers as keys, so with low or no duplicates) using JProfiler and recording CPU time; the profiling used the implementations of BSTs and randomized treaps provided in the book's repo; it's worth remarking that this profiling only gives us information on the implementations we examine, while we could get different results on optimized or anyway differently designed software.

Anyway, the results for our profiling run are shown in figure 3.20, where we can see that for insertion (method `add`) the cumulative running time spent for `RandomizedTreap` is almost twice as much as for `BST::add`; for method `remove`, instead, the ratio becomes 3.5 times.